



Metadata-Driven Data Quality Framework: Scaling Trust Across Enterprise Data Pipelines

Portfolio Case Study 02

A senior data governance and data quality case study focused on transforming fragmented, manual validation into a scalable metadata-driven governance & Data Quality framework.

Focus: Data Governance • Data Quality • DataOps • Metadata Management
• Auditability • Operational Resilience • AI/ML Readiness

Executive Impact

85% Faster

Client onboarding reduced from **4–8 weeks** to **2–3 days**

90%+ Reduction

Duplicate resolution improved from manual, weeks-long effort to automated processing

80%+ Reduction

Incident resolution improved from **20–100+ hours** to **less than 4 hours**

Up to \$5M+ Savings Potential

Annual incident cost reduced from multi-million-dollar exposure to substantially lower operating cost

At a Glance

Category	Details
Industry	Regulated Enterprise / Financial Services-style data environment
Business Area	Customer, financial, operational, analytics, reporting, compliance, and AI/ML data pipelines
Core Problem	Data quality checks were inconsistent, manually maintained, and embedded inside individual pipelines
Solution Pattern	Metadata-driven Data Quality Framework with configurable validation, de-duplication, reconciliation, exception handling, and audit logging
Primary Outcome	Faster onboarding, reduced manual effort, stronger auditability, and trusted Golden Record output
Business Value	Improved data reliability, operational efficiency, governance maturity, and executive confidence in downstream analytics and decisioning

Executive Summary

A regulated enterprise faced growing business risk because poor data quality was slowing onboarding, increasing incident investigation effort, weakening trust in reporting, and creating avoidable operational cost.

I designed a metadata-driven Data Quality Framework that separated validation rules from pipeline code, automated quality checks and de-duplication, generated governed Golden Records, and integrated exception workflows with ServiceNow.

The framework improved onboarding speed by **85%**, reduced duplicate resolution effort by **90%+**, cut incident resolution effort by **80%+**, reduced

debug effort from days or weeks to minutes, and created a reusable governance pattern for scalable enterprise data quality.

The Challenge

Poor data quality was not just a technical inconvenience. It created measurable business drag through lost time, delayed onboarding, duplicate records, manual investigation, and reduced trust in business decisions.

The carousel highlights the scale of the problem: poor data quality can cost organizations **\$2.5M to \$8M annually**, with **40–60% of incidents** traced to bad or duplicate incoming data. Each incident could consume **100+ human hours** of investigation and root-cause analysis, while manual client onboarding could take **4–8 weeks**.

Common issues included:

- Missing mandatory fields
- Invalid date or numeric formats
- Duplicate records
- Negative or out-of-range values
- Special character issues
- Broken source-to-target mappings
- Manual reconciliation gaps
- Lack of audit trail for failed records
- Per-client custom code changes
- Slow root-cause analysis

For a CDO or data governance leader, the risk was clear: fragmented data quality controls were increasing cost, slowing delivery, weakening auditability, and reducing confidence in analytics, reporting, and AI/ML readiness.

Why This Mattered

Data quality became a business performance issue, not just a data engineering issue.

When every client, feed, or data product required custom validation logic, the organization inherited a fragile operating model. Schema changes, duplicate records, and threshold breaches could trigger manual investigation, delay onboarding, and create inconsistent quality outcomes.

The objective was to move from fragmented validation to a reusable governance engine: configure once, govern forever.

My Approach: Standardize → Automate → Govern

Standardize

I centralized validation logic into control-file metadata rather than embedding rules inside each pipeline.

The control file defined field mappings, mandatory attributes, data types, formats, thresholds, survivorship rules, and exception actions. This allowed teams to onboard new clients or data feeds by updating configuration instead of changing pipeline code.

Business value: Validation rules became reusable, scalable, and easier to govern.

Automate

The framework executed validation and de-duplication automatically across incoming files.

It performed schema checks, checksum validation, duplicate file rejection, data type validation, value-range checks, date and email format checks, fuzzy address matching, threshold-based quarantine, exact and fuzzy matching, probabilistic scoring, and survivorship logic for Golden Record creation.

Business value: Manual inspection shifted to automated quality control and exception-based review.

Govern

The framework produced audit-ready evidence and operational visibility.

Failed rows were captured in error reports, duplicate clusters were identified, match scores were retained, threshold gauges showed whether processing could continue, and rejected files could be downloaded for follow-up.

ServiceNow incidents were auto-created, enriched with row numbers and error

reasoning, routed by category, and linked to knowledge articles for faster resolution.

Business value: Data quality became measurable, traceable, and operationally accountable.

Architecture Overview

| **Input** → **Process** → **Output**

Input

Client data, source files, control files, JSON/YAML configurations, field mappings, validation rules, match thresholds, and survivorship rules.

Process

Metadata rules engine parses control files, applies schema validation, maps fields to internal standards, performs validation and de-duplication, applies threshold checks, triggers quarantine or rejection logic, creates ServiceNow incidents, and applies Golden Record survivorship rules.

Output

Accepted records, rejected files, quarantined records, error reports, duplicate clusters, ServiceNow incidents, reconciliation reports, audit logs, and trusted Golden Records.

Key Design Decisions

- **Control-file configuration instead of per-client custom code:** This allowed new structures and rules to be handled through metadata changes, reducing client onboarding from **4–8 weeks** to **3–5 days** and making the core engine reusable.
 - **Automated de-duplication and Golden Record logic instead of manual duplicate resolution:** This reduced duplicate resolution effort by **90%+** and created a single authoritative record with survivorship rules and audit trail.
 - **ServiceNow automation instead of manual incident follow-up:** Auto-created, categorized, and enriched incidents reduced manual effort by **75%** and improved MTTR by **4x**.
-

Results & Business Impact

Impact Area	Before	After	Improvement
Client Onboarding	4–8 weeks	3–5 days	85% faster
Duplicate Resolution	Manual, weeks	Automated	90%+ reduction
Incident Resolution	20–100+ hours	Less than 4 hours	80%+ reduction
Data Debug Effort	Days / weeks	Minutes	80%+ reduction
Client Code Changes	Per-client custom logic	Control file only	100% reusable core engine
Annual Incident Cost	Multi-million-dollar exposure	Substantially lower	Up to \$5M+ savings potential

The solution transformed data quality from a cost center into a strategic governance capability. It improved operational speed, reduced manual investigation, strengthened audit evidence, and created trusted Golden Records for downstream analytics, reporting, and decisioning.

Leadership Takeaway

Better data quality creates better business decisions, stronger trust, and lower operational cost.

This case demonstrates how senior data governance leadership can convert fragmented, manual quality practices into a scalable enterprise control framework.

The solution did not treat data quality as a narrow technical checklist. It treated data quality as a governance capability that supports business trust, audit readiness, operational resilience, cost reduction, and AI/ML readiness.

What I'd Do Differently

If scaling this further, I would add predictive data quality scoring and business-impact weighting so teams could prioritize defects based on downstream risk, not just rule failure counts.

Transferable To

Healthcare: This pattern applies to claims, member eligibility, provider data, clinical reporting, and regulatory submissions because care quality, payment accuracy, and compliance depend on complete and governed data.

Insurance: This pattern applies to policy, claims, underwriting, actuarial feeds, and compliance reporting because quality defects can affect pricing, reserving, risk selection, claims operations, and regulatory confidence.

Portfolio Positioning Statement

This case study positions me as a senior data governance leader who can turn manual, inconsistent validation into a metadata-driven quality framework: improving trust, accelerating onboarding, reducing incident effort, strengthening auditability, and creating reusable governance capabilities across enterprise data pipelines.